

```

250 # Display pivot structure (natural from random matrix)
251 print(f"Matrix rank: {len(pivot_cols)} (number of pivots)")
252 print(f"Free variables: {len(non_pivot_cols)} (columns {non_pivot_cols})")
253
254
255 # Particular solution
256 x_p = particular_solution(rref_result, pivot_cols, n)
257 print(f"\nParticular solution x_p (free vars = 0):")
258 print([round(val, 4) for val in x_p])
259
260 # Null space basis
261 null_basis = nullspace_basis(rref_result, pivot_cols, non_pivot_cols, n)
262 print(f"\nNull space basis (solutions to Ax = 0):")
263 for i, vec in enumerate[Any](null_basis):
264     print(f"  n_{i+1}: {[round(val, 4) for val in vec]}")
265
266 # General solution
267 print(f"\nGeneral solution form:")
268 if len(null_basis) == 1:
269     print(f"  x = x_p + t1·n1")
270 else:
271     print(f"  x = x_p + t1·n1 + ... + t{len(null_basis)}·n{len(null_basis)}")
272 print(f"  where t1, t2, ... are free parameters")
273
274 # Verification
275 print(f"\nVerification:")
276 # Use x4 = 0 (particular solution)
277 x_test = x_p[:]
278 Ax = matrix_vector_multiply(A, x_test)
279 residual = [Ax[i] - b[i] for i in range(len(b))]
280 max_error = max(abs(r) for r in residual)
281
282 print(f"  Test solution (x4 = 0): {[round(val, 3) for val in x_test]}")
283 print(f"  A·x = {[round(val, 3) for val in Ax]}")
284 print(f"  Expected b = {b}")
285 print(f"  Residual ||Ax - b||: {format_number(max_error)}")
286 if max_error < 1e-9:
287     print("  Solution is CORRECT!")
288 else:
289     print("  Solution has errors")
290
291 print("\n" + "=" * 70)
292 print("HANDWRITTEN EXAMPLE COMPLETED - READY FOR SCREENSHOTS")
293 print("=" * 70)
294
295 return rref_result, pivot_cols, non_pivot_cols, x_p, null_basis
296

```

```

197 # =====
198 # MAIN EXECUTION FOR Q1
199 # =====
200
201 def run_handwritten_example():
202     """Run the 3x4 handwritten example for Q1(a) and Q1(b) screenshot purposes"""
203     print("=" * 70)
204     print("Q1(a) & Q1(b): HANDWRITTEN EXAMPLE (3x4 MATRIX)")
205     print("=" * 70)
206
207     # The exact matrix from your handwritten work
208     A = [
209         [1, 2, 1, 1],
210         [2, 5, 3, 2],
211         [1, 1, 2, 1]
212     ]
213     b = [5, 12, 6]
214
215     print("System of equations:")
216     print("x1 + 2x2 + x3 + x4 = 5")
217     print("2x1 + 5x2 + 3x3 + 2x4 = 12")
218     print("x1 + x2 + 2x3 + x4 = 6")
219
220     print_matrix(A, "Original Matrix A", decimals=0)
221     print(f"Vector b: {b}")
222
223     # Create augmented matrix
224     aug = create_augmented_matrix(A, b)
225     print_matrix(aug, "Augmented Matrix [A|b]", decimals=0)
226
227     # Q1(a): REF and RREF
228     print("\n" + "=" * 50)
229     print("Q1(a): REF AND RREF COMPUTATION")
230     print("=" * 50)
231
232     ref_result = ref(aug)
233     print_matrix(ref_result, "REF [A|b]", decimals=3)
234
235     rref_result = rref(aug)
236     print_matrix(rref_result, "RREF [A|b]", decimals=3)
237
238     # Q1(b): Pivot analysis and solutions
239     print("\n" + "=" * 50)
240     print("Q1(b): PIVOT ANALYSIS AND SOLUTIONS")
241     print("=" * 50)
242
243     n = len(A[0]) # number of variables
244     pivot_cols, non_pivot_cols = find_pivot_columns(rref_result)
245

```

```

141 def nullspace_basis(rref_matrix, pivot_cols, non_pivot_cols, n):
142     """
143     Find basis vectors for the null space (solutions to  $Ax = 0$ )
144     One basis vector for each free variable
145     """
146     null_basis = []
147
148     for free_col in non_pivot_cols:
149         # Create vector: set this free variable to 1, others to 0
150         null_vec = [0.0] * n
151         null_vec[free_col] = 1.0
152
153         # For each pivot, read off the negative of the free column's coefficient
154         for i, pivot_col in enumerate(Any](pivot_cols):
155             if i < len(rref_matrix):
156                 null_vec[pivot_col] = -rref_matrix[i][free_col]
157
158         null_basis.append(null_vec)
159
160     return null_basis
161
162
163 def matrix_vector_multiply(A, x):
164     """Multiply matrix A with vector x"""
165     m = len(A)
166     result = []
167     for i in range(m):
168         val = sum(A[i][j] * x[j] for j in range(len(x)))
169         result.append(val)
170     return result
171
172
173 def verify_solution(A, x, b):
174     """Check if  $Ax$  equals  $b$  (within tolerance)"""
175     Ax = matrix_vector_multiply(A, x)
176     residual = [Ax[i] - b[i] for i in range(len(b))]
177     max_error = max(abs(r) for r in residual)
178     return max_error < 1e-9, max_error
179
180

```

```

99 def find_pivot_columns(rref_matrix):
100     """
101     Identify pivot and non-pivot columns from RREF
102     Returns: (pivot_cols, non_pivot_cols)
103     """
104     m = len(rref_matrix)
105     n = len(rref_matrix[0]) - 1 # Exclude augmented column
106
107     pivot_cols = []
108
109     for row in range(m):
110         for col in range(n):
111             if abs(rref_matrix[row][col] - 1.0) < 1e-9: # Found a leading 1
112                 # Check if this is truly the first non-zero in the row
113                 is_pivot = True
114                 for c in range(col):
115                     if abs(rref_matrix[row][c]) > 1e-9:
116                         is_pivot = False
117                         break
118                 if is_pivot:
119                     pivot_cols.append(col)
120                     break
121
122     non_pivot_cols = [col for col in range(n) if col not in pivot_cols]
123
124     return pivot_cols, non_pivot_cols
125
126
127 def particular_solution(rref_matrix, pivot_cols, n):
128     """
129     Find particular solution by setting all free variables to 0
130     """
131     x_p = [0.0] * n
132
133     # For each pivot column, read the value from the augmented part
134     for i, col in enumerate(Any(pivot_cols)):
135         if i < len(rref_matrix):
136             x_p[col] = rref_matrix[i][-1] # Last column is b
137
138     return x_p
139

```

```

62 def rref(aug_matrix):
63     """
64     Convert augmented matrix to Reduced Row Echelon Form (RREF)
65     First gets REF, then scales pivots to 1 and clears above
66     """
67     matrix = ref(aug_matrix) # Start with REF
68     m = len(matrix)
69     n = len(matrix[0])
70
71     pivot_positions = []
72
73     # Find pivot positions and scale them to 1
74     for row in range(m):
75         pivot_col = -1
76         for col in range(n - 1):
77             if abs(matrix[row][col]) > 1e-9:
78                 pivot_col = col
79                 break
80
81         if pivot_col != -1:
82             pivot_positions.append((row, pivot_col))
83             pivot_val = matrix[row][pivot_col]
84             # Scale entire row so pivot becomes 1
85             for c in range(n):
86                 matrix[row][c] = matrix[row][c] / pivot_val
87
88     # Clear above each pivot
89     for row, col in pivot_positions:
90         for r in range(row):
91             if abs(matrix[r][col]) > 1e-9:
92                 factor = matrix[r][col]
93                 for c in range(n):
94                     matrix[r][c] = matrix[r][c] - factor * matrix[row][c]
95
96     return matrix
97

```

```
8
9 def create_augmented_matrix(A, b):
0     """Combine A and b into augmented matrix [A|b]"""
1     m = len(A)
2     aug = []
3     for i in range(m):
4         row = A[i][:] + [b[i]]
5         aug.append(row)
6     return aug
7
```

```

18 def ref(aug_matrix):
19     """
20     Convert augmented matrix to Row Echelon Form (REF)
21     Returns the REF matrix
22     """
23     matrix = [row[:] for row in aug_matrix] # Deep copy
24     m = len(matrix)
25     n = len(matrix[0])
26
27     current_row = 0
28
29     for col in range(n - 1): #need not pivot on last column (augmented part)
30         # Find pivot - look for non-zero entry at or below current_row
31         pivot_row = -1
32         for row in range(current_row, m):
33             if abs(matrix[row][col]) > 1e-9: # Tolerance for zero
34                 pivot_row = row
35                 break
36
37         # If no pivot found in this column, move to next column
38         if pivot_row == -1:
39             continue
40
41         # Swap rows if needed
42         if pivot_row != current_row:
43             matrix[current_row], matrix[pivot_row] = matrix[pivot_row], matrix[current_row]
44
45         pivot = matrix[current_row][col]
46
47         # Eliminate all entries below the pivot
48         for row in range(current_row + 1, m):
49             if abs(matrix[row][col]) > 1e-9:
50                 factor = matrix[row][col] / pivot
51                 for c in range(n):
52                     matrix[row][c] = matrix[row][c] - factor * matrix[current_row][c]
53
54         current_row += 1
55         if current_row >= m:
56             break
57
58     return matrix
59
60

```

```

298 def main():
394     print("\n" + "=" * 70)
395     print("Q1 completed successfully!")
396     print("=" * 70)
397
398     print("\n" + "=" * 70)
399     print("VERIFICATION CHECKLIST FOR HANDWRITTEN SUBMISSION")
400     print("=" * 70)
401
402     print("\nQ1(a): REF and RREF Implementation [2 marks]")
403     print("    Hand example: 3x3 or 3x4 augmented matrix reduced to REF")
404     print("    Hand example: Same matrix reduced to RREF")
405     print("    Code: ref() function (handwritten/screenshot)")
406     print("    Code: rref() function (handwritten/screenshot)")
407     print("    Output: REF computed successfully (staircase structure)")
408     print("    Output: RREF computed successfully (unit-pivot form)")
409
410     print("\nQ1(b): Pivot Analysis and Solutions [1 mark]")
411     print("    Code: find_pivot_columns() function")
412     print("    Code: particular_solution() function")
413     print("    Code: nullspace_basis() function")
414     print("    Output: Functions working correctly")
415     print("    Comment: Free columns are handled as parameters, not solved directly")
416
417     print("\nQ1(c): Random 6x9 System [2 marks]")
418     print("    Random 6x9 matrix A generated")
419     print("    Vector b generated (consistent system)")
420     print("    REF computed and displayed")
421     print("    RREF computed and displayed")
422     print(f"    Pivot columns identified: {len(pivot_cols)} pivots")
423     print(f"    Non-pivot columns identified: {len(non_pivot_cols)} free variables")
424     print("    Particular solution x_p computed")
425     print(f"    Null space basis computed: {len(null_basis)} vectors")
426     print("    General solution form displayed")
427     print(f"    Verification: Residual = {format_number(error)} - Excellent!")
428     print("    Interpretation: The small residual means the answer checks out numerically")
429
430     print("\n" + "=" * 70)
431
432
433 if __name__ == "__main__":
434     main()
435

```

```

298 def main():
347     # Compute RREF
348     rref_result = rref(aug)
349     print_matrix(rref_result, "RREF [A|b]", decimals=3)
350
351     # Now using the RREF, we can read which columns have pivots.
352     pivot_cols, non_pivot_cols = find_pivot_columns(rref_result)
353     print(f"\nPivot columns (indices): {pivot_cols}")
354     print(f"Non-pivot columns (free variables, indices): {non_pivot_cols}")
355     print(f"Number of pivots: {len(pivot_cols)}")
356     print(f"Number of free variables: {len(non_pivot_cols)}")
357     print("So columns without pivots become the free-variable columns.")
358     print("This is why we get three parameters in the final answer.")
359
360     # For the particular solution, I keep the free variables as zero.
361     x_p = particular_solution(rref_result, pivot_cols, n)
362     print(f"\nParticular solution x_p (free vars = 0):")
363     print([round(val, 4) for val in x_p])
364
365     # For Ax = 0, take one free variable as 1 at a time.
366     null_basis = nullspace_basis(rref_result, pivot_cols, non_pivot_cols, n)
367     print(f"\nNull space basis (solutions to Ax = 0):")
368     for i, vec in enumerate[Any](null_basis):
369         print(f"  n_{i+1}: {[round(val, 4) for val in vec]}")
370
371     # Hence, all solutions are one particular solution plus null-space directions.
372     print("\nGeneral solution form:")
373     print(f"x = x_p + t1·n1 + t2·n2 + ... + t{len(null_basis)}·n{len(null_basis)}")
374     print(f"where t1, t2, ..., t{len(null_basis)} are free parameters")
375
376     # Now I am checking one random choice of the free parameters.
377     print("\nVerification:")
378     # Generate random coefficients for the free parameters
379     coeffs = []
380     for i in range(len(null_basis)):
381         coeffs.append(rng.uniform(-1.0, 1.0))
382     x_test = x_p[:]
383     for i, coeff in enumerate[Any](coeffs):
384         for j in range(n):
385             x_test[j] += coeff * null_basis[i][j]
386
387     is_valid, error = verify_solution(A, x_test, b)
388     print(f"Random coefficients t: {[round(c, 3) for c in coeffs]}")
389     print(f"Test solution x: {[round(val, 3) for val in x_test]}")
390     print(f"Residual ||Ax - b||: {format_number(error)}")
391     print(f"Solution valid: {'YES' if is_valid else 'NO'}")
392     print("Since this residual is almost zero, the computed x satisfies Ax = b.")
393
394     print("\n" + "=" * 70)
395     print("Q1 completed successfully!")
396     print("\n" + "=" * 70)

```

```

297
298 def main():
299     # First run the handwritten example for Q1(a) and Q1(b)
300     run_handwritten_example()
301
302     print("\n\n")
303     print("=" * 70)
304     print("Q1(c): RANDOM 6x9 MATRIX SYSTEM")
305     print("=" * 70)
306     print("Now using the functions for the required random 6x9 example in Q1(c).")
307
308     # Q1(c): Random 6x9 system
309     print("\n" + "=" * 70)
310     print("Q1(c): Random 6x9 Matrix System")
311     print("=" * 70)
312
313     m, n = 6, 9
314
315     # Set random seed for reproducibility (as per guidelines)
316     rng = random.Random(42)
317
318     # Generate random 6x9 matrix A (following Q1(c) hint: only entries are random)
319     print("Generating random 6x9 matrix A with random entries...")
320
321     # Simple random generation as specified in the hint
322     A = [[0.0 for _ in range(n)] for _ in range(m)]
323     for i in range(m):
324         for j in range(n):
325             A[i][j] = float(rng.randint(-3, 3))
326
327     # Generate random seed vector x_seed to ensure consistent system
328     print("Generating random seed vector x_seed...")
329     x_seed = []
330     for i in range(n):
331         x_seed.append(float(rng.randint(0, 3)))
332
333     # Compute b = A x x_seed to guarantee the system Ax = b has a solution
334     print("Computing b = A x x_seed to ensure consistency...")
335     b = matrix_vector_multiply(A, x_seed)
336
337     print_matrix(A, "Random 6x9 Matrix A", decimals=2)
338     print(f"\nVector b: {[round(val, 2) for val in b]}")
339
340     # Create augmented matrix
341     aug = create_augmented_matrix(A, b)
342
343     # Compute REF
344     ref_result = ref(aug)
345     print_matrix(ref_result, "REF [A|b]", decimals=3)
346

```